Topic 8: Functions

7.1 What is a Function?

- A function is a named, reusable block of code that performs a specific task
- Write once, call many times from anywhere in the program
- Functions make programs organized, readable, and easier to debug
- C++ programs are built from functions — `main()` itself is a function

7.2 Function Declaration and Definition

```
int add(int a, int b);   // declaration (prototype)

int main() {
    cout << add(5, 3);   // call
}

int add(int a, int b) {  // definition
    return a + b;
}
```

- Declaration (prototype): tells the compiler the function exists before it is defined
- If the definition appears before main(), no separate declaration is needed

7.3 Parameters and Return Values

- Parameters: variables listed in the function definition — `int a, int b`
- Arguments: actual values passed when calling — `add(5, 3)`
- Return type before the function name — `int add(...)` returns an int
- `return` sends the value back to the caller and exits the function
- `void` functions perform an action but return nothing

7.4 Default Parameters

```
void greet(string name, string lang = "C++") {
    cout << name << " is learning " << lang << endl;
}
```

```
greet("Ahmad");          // uses default: C++
greet("Sara", "Python"); // overrides default
```

- Default parameters must be at the rightmost positions

7.5 Function Overloading

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

- Same function name, different parameter types or count
- Compiler picks the correct version based on arguments

7.6 Scope and the Call Stack

- Local variable: declared inside a function — only accessible within that function
- Global variable: declared outside all functions — accessible everywhere (avoid overuse)
- Call stack: when main() calls add(), add()'s frame is pushed; when it returns, it is popped
- Stack frame holds: the function's local variables, parameters, and return address

---